

Learning Decomposed Contextual Token Representations from Pretrained and Collaborative Signals for Generative Recommendation

Yifan Liu*
University of Illinois
Urbana-Champaign
Champaign, Illinois, USA
yifan40@illinois.edu

Yaokun Liu*
University of Illinois
Urbana-Champaign
Champaign, Illinois, USA
yaokunl2@illinois.edu

Zelin Li
University of Illinois
Urbana-Champaign
Champaign, Illinois, USA
zelin3@illinois.edu

Zhenrui Yue
University of Illinois
Urbana-Champaign
Champaign, Illinois, USA
zhenrui3@illinois.edu

Gyuseok Lee
University of Illinois
Urbana-Champaign
Champaign, Illinois, USA
gyuseok2@illinois.edu

Ruichen Yao
University of Illinois
Urbana-Champaign
Champaign, Illinois, USA
ryao8@illinois.edu

Yang Zhang
Miami University
Miami, Florida, USA
zhang981@miamioh.edu

Dong Wang
University of Illinois
Urbana-Champaign
Champaign, Illinois, USA
dwang24@illinois.edu

Abstract

Recent advances in generative recommenders adopt a two-stage paradigm: items are first tokenized into semantic IDs using a pretrained tokenizer, and then large language models (LLMs) are trained to generate the next item via sequence-to-sequence modeling. However, these two stages are optimized for different objectives: semantic reconstruction during tokenizer pretraining versus user interaction modeling during recommender training. This objective misalignment leads to two key limitations: (i) suboptimal static tokenization, where fixed token assignments fail to reflect diverse usage contexts; and (ii) discarded pretrained semantics, where pretrained knowledge—typically from language model embeddings—is overwritten during recommender training on user interactions. To address these limitations, we propose to learn **DE**composed **CO**ntextual **T**oken **R**epresentations (DECOR), a unified framework that preserves pretrained semantics while enhancing the adaptability of token embeddings. DECOR introduces contextualized token composition to refine token embeddings based on user interaction context, and decomposed embedding fusion that integrates pretrained codebook embeddings with newly learned collaborative embeddings. Experiments on three real-world datasets demonstrate that DECOR consistently outperforms state-of-the-art baselines in recommendation performance. Our code is available at ¹.

*Both authors contributed equally to this research.

¹<https://github.com/yliuuaa/DECOR.git>



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGIR '26, Melbourne, VIC, Australia*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2599-9/2026/07

<https://doi.org/10.1145/3805712.3809578>

CCS Concepts

• **Information systems** → **Language models**; **Recommender systems**; *Personalization*.

Keywords

Generative Recommendation, Sequential Recommendation, Item Tokenization

ACM Reference Format:

Yifan Liu, Yaokun Liu, Zelin Li, Zhenrui Yue, Gyuseok Lee, Ruichen Yao, Yang Zhang, and Dong Wang. 2026. Learning Decomposed Contextual Token Representations from Pretrained and Collaborative Signals for Generative Recommendation. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '26)*, July 20–24, 2026, Melbourne, VIC, Australia. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3805712.3809578>

1 Introduction

Recent advances in Large Language Models (LLMs) have led to the emergence of *generative recommendation* as a new paradigm for sequential recommendation [4, 20]. Numerous efforts investigating generative recommenders have formulated recommendations as an auto-regressive sequence generation task, aligning naturally with the strengths of LLMs in modeling long-range dependencies and generating coherent sequences [4, 17, 24, 30]. Additionally, generative recommender systems naturally possess the advantage of handling cold-start items by leveraging pre-training knowledge to provide semantic priors and contextual understanding. Specifically, generative recommenders typically follow a two-stage training pipeline: *item tokenization* and *recommender training*. In the item tokenization stage, item metadata (e.g., name, description) is first encoded into pretrained semantic embeddings. These semantic embeddings are then tokenized by a pretrained tokenizer, which is

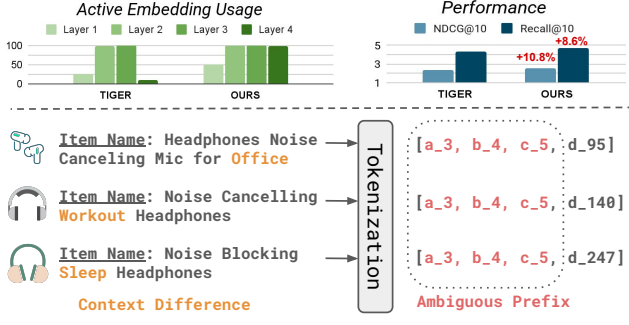


Figure 1: Suboptimal static tokenization assigns identical prefix tokens to semantically distinct items (e.g., noise-canceling headphones for office use, workout, or sleep), leading to ambiguous representations that fail to reflect diverse user interaction contexts.

often a vector quantizer [13, 23] with a fixed vocabulary trained to reconstruct the original embeddings. The item token sequences, also known as semantic IDs, are cached and then used to train an LLM (e.g., T5 [19]) to predict the recommended next item.

The two-stage generative recommenders introduce an objective misalignment: the tokenizer learns to encode pretrained semantics and produces static token representations (semantic IDs) for items that remain fixed during recommender training, while the recommender is optimized for sequential behavior modeling. The two-stage objective misalignment prevents the static token representations from adapting to diverse recommendation contexts, and the recommender training erodes the pretrained semantic knowledge captured during tokenization. Specifically, we aim to address two limitations as follows:

(C1) Suboptimal Static Tokenization: Semantic IDs are produced by a pretrained tokenizer trained to reconstruct semantic embeddings rather than to optimize recommendation performance, leading to suboptimal tokenization for recommendation [20, 24].

In particular, static tokenization imposes a representation bottleneck for recommenders that forces behaviorally distinct items to share identical representations for 75% of the generation steps (prefix: [a_3, b_4, c_5]), blinding the model to user intent until the final token generation. The consequent lack of discriminative input limits the attention mechanism, which fails to resolve user intent from structurally identical representations. As illustrated in Figure 1, items intended for different purposes (e.g., sleep, office, or workout use) receive identical prefix tokens, leading to *prefix ambiguity*. While recent efforts have sought to address this issue by jointly optimizing the tokenizer and the recommender [17, 27], such approaches require repeated re-tokenization throughout training, which introduces instability and increases computational overhead.

(C2) Discarded Pretrained Semantics: During recommender training, the semantic knowledge encoded in pretrained embeddings is discarded after tokenization. While semantic IDs are trained to reconstruct pretrained semantic embeddings, the corresponding token embeddings are randomly initialized and trained solely on user interaction data. This two-stage design forfeits the semantics captured by the pretrained model (e.g., world knowledge in LLM

embeddings). For example, token *a_54* is pretrained to represent both the fruit and the technology brand meanings of “apple”. However, if *a_54* appears predominantly in electronics-related contexts during training, its embedding captures only the tech brand, leading to misaligned recommendations for sparse items such as suggesting laptops to users seeking groceries.

While recent efforts have attempted to overcome these issues (C1 and C2), they remain limited in important ways. For example, ETE-GRec [17] jointly optimizes the tokenizer and recommender in an end-to-end fashion with repeated re-tokenization during training, which leads to instability and results in suboptimal convergence (as illustrated in Figure 5). Other methods, such as LETTER [24] and CoST [32], aim to enhance the semantic knowledge preservation of item tokenization in the pretraining stage. However, the enhanced static tokenization still discards the fine-grained semantic information in pretrained embeddings in the recommender training stage, limiting the ability to leverage external knowledge. In contrast, our work provides a novel solution that avoids re-tokenization while retaining pretrained semantics, yielding a more stable yet expressive generative recommender framework.

Specifically, we propose **DECOR** (Decomposed Contextual Token Representations), a novel framework that contextually adapts token embeddings with pretrained semantic embeddings and collaborative signals in user interaction sequential modeling. DECOR consists of two key components: *Contextualized Token Composition* and *Decomposed Embedding Fusion*. Specifically, *Contextualized Token Composition* is a lightweight, dynamic interpretation mechanism applied during the recommender model’s token generation to address the suboptimality of static tokenization (C1). Instead of relying solely on the embeddings corresponding to static semantic IDs (as shown in Figure 1), each token embedding is contextually enhanced via a soft composition with token embeddings in a predefined candidate token set, conditioned on the user’s interaction history. This contextual composition of tokens allows the LLM recommender to reinterpret token semantics in a context-dependent manner, compensating for the suboptimality of fixed token assignments. To preserve pretrained semantics (C2), we retain the pretrained codebooks from the RQ-VAE tokenizer as frozen semantic embeddings and introduce separate, learnable collaborative embeddings. These two representations are fused through a light-weight fusion network guided by the recommendation loss, enabling the model to adaptively integrate pretrained semantic knowledge from the frozen codebook and interaction patterns learned during sequential modeling. Our contributions are listed as follows:

- We focus on two key limitations in existing generative recommenders: (i) the *suboptimality of static semantic tokenization*, and (ii) the *discarded pretrained semantics*. We are the first to explicitly analyze how static tokenization limits generative recommender’s ability to adapt item representations to diverse user context dynamics in generative recommendation.
- We propose **DECOR** (Decomposed Contextual Token Representation Learning), a novel framework that fuses frozen pretrained semantic embeddings with learnable collaborative embeddings, and introduces a dynamic, context-aware token composition mechanism to dynamically interpret static input tokens.

- We conduct comprehensive experiments on three real-world datasets, demonstrating that DECOR consistently outperforms recent baselines. Notably, DECOR achieves faster convergence than the end-to-end tokenization–recommender joint training baseline while delivering higher accuracy with only moderate computational overhead.

2 Related Works

2.1 Generative Recommender

In recent years, generative recommendation has emerged as a promising paradigm that formulates the sequential recommendation task as a sequence-to-sequence problem and directly generates the unique identifier of the next item [4]. Early work such as P5 [6] fine-tunes a pretrained language model (e.g., T5 [19]) to handle multiple recommendation tasks within a unified generative framework. TIGER [20] advances this paradigm by introducing a discrete semantic tokenization scheme based on item metadata (e.g., title, description), enabling the use of a pretrained T5 to autoregressively generate item token sequences. Building on TIGER, EAGER [26] proposes a two-stream generation framework with a shared encoder and separate decoders to jointly capture user behavior and item content semantics. OneRec [5] further extends the generative approach by unifying retrieval and ranking into a single iterative generation process. Our work investigates the under-explored limitations of the tokenization pretraining–recommender training process in generative recommenders, addressing the suboptimality of static tokenization and the loss of pretrained semantics during training.

2.2 Item Token Representation

A key step in generative recommenders is to represent items as discrete tokens that can be consumed by language models. Early approaches fall into two broad categories: pseudo ID-based and text-based. To encode a large number of items, pseudo ID-based methods assign each item a unique token identifier without incorporating any semantic structure [2, 6, 10, 25]. Text-based methods, on the other hand, utilize item metadata (e.g., titles, descriptions) to construct natural language prompts representing input item sequences [1, 3, 14–16, 28, 29]. While more expressive, text-based methods incur high inference costs and may introduce hallucinated content [10]. To balance semantic fidelity and token efficiency, recent work introduces semantic indexing schemes that extract compact token sequences by quantizing pretrained text embeddings of item descriptions [7, 20, 24, 30, 32]. However, existing semantic indexers adopt static tokenization, where tokenized item representations remain fixed throughout training, limiting the model’s ability to adapt item representations based on recommendation signals. To address the static tokenization limitation, ETEGRec [17] jointly trains the item tokenizer and recommender model in an end-to-end fashion, though this coupling may introduce instability as token assignments evolve during training. ED² [27] proposes a duo-index framework with a multi-grained token regulator and instruction tuning with user-level metadata, which may not be available in all scenarios. In contrast, our method enables stable yet contextually adaptive token interpretation during generation without relying on user-specific information. Moreover, a recent

concurrent work, ActionPiece [9], models context by applying BPE-based tokenization to group frequent item transitions into semantic units, enabling a standard Transformer to predict the next “action piece” via self-attention without requiring auxiliary parameter modules. Our work keeps the next-item prediction formulation with a context-enhanced item token representation.

3 Method

3.1 Problem Formulation

We consider the generative recommendation task under the sequential recommendation scenario. Formally, given the set of items $\mathcal{I}$ and a user’s interaction sequence $\mathbf{S}^u = [i_1, i_2, \dots, i_{t-1}] \in \mathcal{I}$, the generative recommendation task is to predict the next item $i_t \in \mathcal{I}$. Generative recommendation addresses this task through two key steps: item tokenization and autoregressive generation. Item tokenization maps each item $i \in \mathcal{I}$ into a discrete token sequence $\mathbf{c}_i = [c_{i,1}, c_{i,2}, \dots, c_{i,L}] \in \mathcal{C}$, where L is the sequence length and $\mathcal{C}$ is predefined token set. The interaction sequence $\mathbf{S}^u$ is thereby transformed into a tokenized sequence $\mathbf{X}^u = [\mathbf{c}_{i_1}, \mathbf{c}_{i_2}, \dots, \mathbf{c}_{i_{t-1}}]$. Given $\mathbf{X}^u$, the model autoregressively generates the token sequence $\mathbf{c}_{i_t}$ corresponding to the next target item i_t by factorizing the conditional probability:

$$p(\mathbf{c}_{i_t} | \mathbf{X}^u) = \prod_{l=1}^L p(c_{i_t,l} | \mathbf{X}^u, c_{i_t,1}, \dots, c_{i_t,l-1}). \quad (1)$$

3.2 Semantic Indexer Pretraining

As shown in the item tokenization pretraining stage of Figure 2, we use RQ-VAE [20] as our semantic indexer, which consists of a pair of MLP encoder-decoder, and a sequence of codebooks $\{\mathbf{C}_1, \dots, \mathbf{C}_M\}$ where M is the number of quantization levels for residual quantization. Given a pretrained embedding of an item $\mathbf{x} \in \mathcal{R}^d$, the goal of a semantic indexer is to encode the pretrained semantics into a sequence of discrete tokens named semantic ID. Empirically, the length of the semantic ID obtained is $M + 1$, where an additional token is appended to the output of the semantic indexer in order to resolve any duplicates. We denote the complete semantic ID length as $M_s = M + 1$ in the subsequent discussions. The semantic indexing operates first by down-projecting the pretrained embedding vector to a latent vector $\mathbf{z}_0$ by an encoder network.

$$\mathbf{z}_0 = \text{Encoder}(\mathbf{x}). \quad (2)$$

Following the encoder projection, the hierarchical quantization is performed through M residual steps:

$$\mathbf{z}_m = \mathbf{z}_{m-1} - q_l(\mathbf{z}_{m-1}), \quad m = 1, \dots, M \quad (3)$$

$$q_m(\mathbf{z}_{m-1}) = \arg \min_{\mathbf{e} \in \mathbf{C}_m} \|\mathbf{z}_{m-1} - \mathbf{e}\|_2 \quad (4)$$

where $q_l(\cdot)$ performs nearest-neighbor lookup with Euclidean distance in codebook $\mathbf{C}_m$.

The quantized semantic ID of an item is the concatenation of code indices $\{i_1, \dots, i_{M_s}\}$ of the nearest vectors in codebooks from each level, with the final token for item collision handling. In particular, we adopt the collision handling scheme of [20], which resolves items sharing the same first M semantic IDs by appending incremental indices. For example, as shown in Figure 2, an item may be assigned

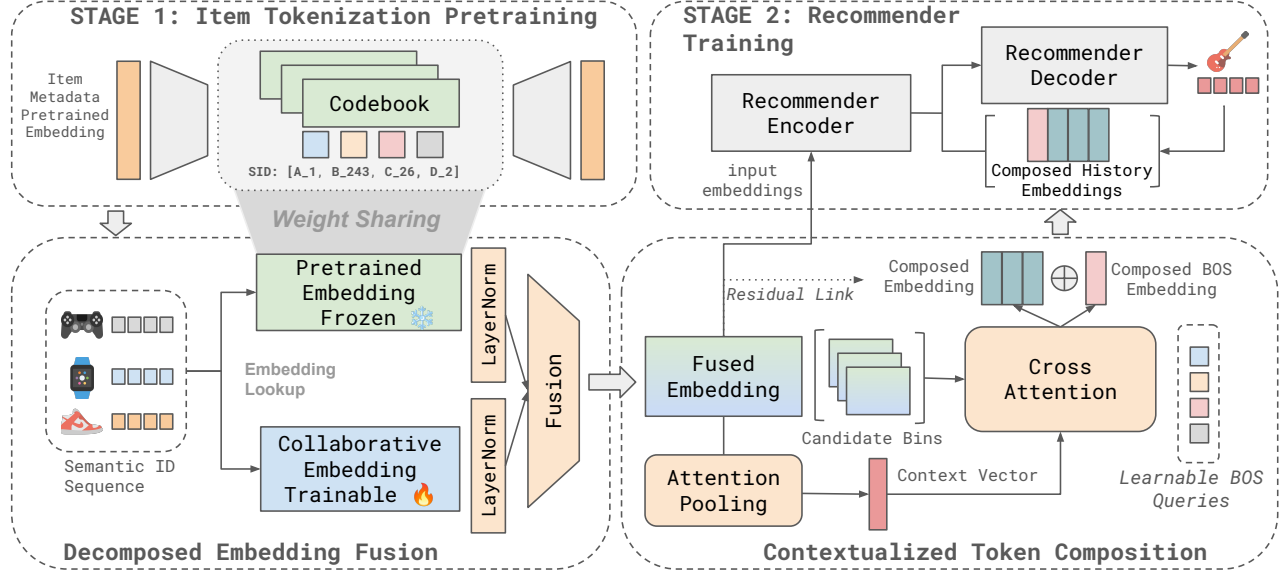


Figure 2: DECOR enhances generative recommendation via two components: decomposed embedding fusion integrates frozen pretrained embeddings and newly learned collaborative embeddings, while contextualized token composition dynamically refines token representations during autoregressive generation.

the semantic ID [A_1, B_243, C_26, D_2], with the character prefixes included for clarity of demonstration.

The training of RQ-VAE semantic indexer is under the self-supervision of pretrained semantic embedding reconstruction. To perform reconstruction, each item’s chosen codebook vectors are summed up, resulting in a codebook-estimated representation $\tilde{\mathbf{r}} = \sum_{m=1}^M \mathbf{e}_{c_m}$. The estimated representation $\tilde{\mathbf{r}}$ is then decoded back to semantic space where $\mathbf{e}_{c_m}$ is the corresponding codebook vector to semantic ID token i_m in codebook c_m :

$$\tilde{\mathbf{x}} = \text{Decoder}(\tilde{\mathbf{r}}). \quad (5)$$

With the reconstructed representation $\tilde{\mathbf{x}}$, the whole RQ-VAE semantic indexer is optimized with $\mathcal{L}_{\text{RQ}}$ where:

$$\mathcal{L}_{\text{RQVAE}} = \mathcal{L}_{\text{RECON}} + \mathcal{L}_{\text{RQ}} \quad (6)$$

$$\mathcal{L}_{\text{RECON}} = \|\mathbf{x} - \tilde{\mathbf{x}}\|_2^2 \quad (\text{reconstruction}) \quad (7)$$

$$\mathcal{L}_{\text{RQ}} = \sum_{m=1}^M \underbrace{\|\text{sg}[\mathbf{z}_{m-1}] - \mathbf{e}_{c_m}^m\|_2^2}_{\text{codebook update}} + \beta \underbrace{\|\mathbf{z}_{m-1} - \text{sg}[\mathbf{e}_{c_m}^m]\|_2^2}_{\text{commitment}} \quad (8)$$

where $\text{sg}[\cdot]$ denotes stop-gradient operation, and $\beta = 0.25$ balances codebook learning [13]. The reconstruction loss $\mathcal{L}_{\text{RECON}}$ ensures the reconstructed semantic embedding retains the original semantic meaning, and the commitment loss $\mathcal{L}_{\text{RQ}}$ encourages proximity between residual vectors and codebook embeddings.

3.3 Decomposed Embedding Fusion

To retain the rich semantic information (C2) in pretrained semantic embeddings, we introduce a *Decomposed Embedding Fusion* module that dynamically fuses pretrained semantic and newly learned collaborative embedding representations. We treat the two modalities

as complementary information channels and perform modality-aware fusion to adaptively integrate knowledge from both sources.

Specifically, for pretrained semantics, we utilize the codebooks from a pretrained RQ-VAE tokenizer, which provides token-level representations learned to reconstruct pretrained embeddings while preserving the hierarchical structure of semantic IDs introduced by multi-stage tokenization [20]. Formally, we define the pretrained semantic embedding space for decomposed embedding fusion as:

$$E_{\text{pre}} = \{\mathbf{C}_i\}_{i=1}^M \in \mathbb{R}^{K \cdot M \times d}, \quad (9)$$

where each $\mathbf{C}_i \in \mathbb{R}^{K \times d}$ is a frozen codebook of K embeddings for the i -th layer of an M -layer RQ-VAE semantic indexing scheme, and d is the embedding dimension. To ensure compatibility with the downstream LLM recommender, we pretrain the tokenizer using codebooks with the same hidden size d as the recommender. The pretrained semantic embedding $\mathbf{e}_c$ for a token index c_i is retrieved by a direct lookup from the corresponding codebook: $\mathbf{e}_c = \mathbf{C}_i[c_i]$. In parallel, we define the collaborative embedding space $E_{\text{collab}} \in \mathbb{R}^{K \cdot M \times d}$ as a learnable embedding matrix trained from scratch via the autoregressive generation objective. Unlike the pretrained semantic embeddings guided by the reconstruction objective, collaborative embeddings are supervised purely based on user interaction sequences, allowing the model to encode sequential patterns such as co-occurrence and user preference dynamics. Notably, both E_{pre} and E_{collab} share the same dimensionality ($K \times d$), which enables seamless alignment and fusion in subsequent stages.

To bridge the modality gap between pretrained semantics and collaborative user interaction patterns [18, 30], we first project both embeddings into a shared latent space, followed by layer normalizations:

$$\hat{\mathbf{e}}_{\text{pre}} = \text{LN}(W_{\text{pre}} \mathbf{e}_{\text{pre}}), \quad \hat{\mathbf{e}}_{\text{collab}} = \text{LN}(W_{\text{collab}} \mathbf{e}_{\text{collab}}), \quad (10)$$

where $e_{\text{pre}}, e_{\text{collab}} \in \mathbb{R}^d$ are token embeddings from the pretrained and collaborative modalities respectively, $W_{\text{pre}}, W_{\text{collab}} \in \mathbb{R}^{d' \times d}$ are learnable projection matrices, and LN denotes layer normalization. We then concatenate the normalized embeddings and apply a fusion layer to map them back to the original latent space:

$$e_{\text{fused}} = W_{\text{fuse}} [\hat{e}_{\text{pre}} \parallel \hat{e}_{\text{collab}}] \in \mathbb{R}^d, \quad (11)$$

where $W_{\text{fuse}} \in \mathbb{R}^{d \times 2d'}$ is a learnable fusion matrix, and $\parallel$ denotes vector concatenation. This fusion process enables the model to integrate both pretrained semantic and collaborative signals into a unified representation, aligning heterogeneous modalities while preserving their complementary strengths for downstream recommendation. During the forward pass, the decomposed embedding fusion module dynamically computes the fused embedding e_{fused} given the input token sequence c_i for each item i .

3.4 Contextualized Token Composition

In two-stage generative recommendation paradigm, the static tokenization introduces a misalignment between an item’s static semantic ID and an item’s dynamic latent representation learned for recommendation. Ideally, the semantic ID of an item should be dynamic and context-aware—capable of capturing pretrained semantic similarity for unseen items while also adapting to collaborative user interaction patterns learned. In particular, we identify the suboptimality of static tokenization (C1) as illustrated in Figure 1, where multiple items are assigned identical tokens despite differing significantly in their usage contexts.

To better align tokenization to the recommendation objective, recent work introduces an end-to-end training framework where the tokenizer is jointly optimized with the LLM recommender in an iterative manner [17]. However, the alternative optimizations of the tokenizer and recommender introduce training instability and lead to less-efficient training sessions. Instead of dynamically updating the semantic ID tokens of items, we propose to adapt token embeddings through composition with the embeddings of other tokens, enhancing token contextual expressiveness without iteratively performing re-tokenization.

To overcome the limitations of static item tokens, we introduce a contextualized token composition mechanism that refines token embeddings according to usage context. More formally, given a target item i with a cached semantic ID sequence $c_i = \{c_i^{(j)}\}_{j=1}^M$ and a historical context sequence $\mathbf{h}_i$, we compute the context-aware embedding for each token $c \in c_i$ using a function $\Phi(c, u_c, \{e_{c'}\}_{c' \in \mathcal{N}(c)})$, where u_c is a context vector derived from the history and $\mathcal{N}(c)$ is a set of candidate composition tokens for c :

$$\tilde{e}_c = \Phi(c, u_c, \{e_{c'}\}_{c' \in \mathcal{N}(c)}). \quad (12)$$

Context Vector Computation. To obtain the context vector $u_c \in \mathbb{R}^d$ used for generating a specific target token c , we aggregate the fused embeddings of the historical context sequence $\mathbf{h}_c = \{h_1, h_2, \dots, h_L\}$, where each $h_\ell = f_{\text{fuse}}(c_\ell) \in \mathbb{R}^d$ denotes the fused embedding of token c_ℓ obtained via the Decomposed Embedding Fusion module. Specifically, we apply an attention-based pooling mechanism to produce a summary of the context:

$$u_c = \text{AttnPool}(\mathbf{h}_c) = \text{MLP}_{\text{ctx}} \left(\sum_{\ell=1}^L \alpha_\ell \cdot h_\ell \right), \quad (13)$$

where the attention weights $\{\alpha_\ell\}_{\ell=1}^L$ are computed via:

$$s_\ell = \mathbf{w}^\top \tanh(\mathbf{W}h_\ell + \mathbf{b}) \quad (14)$$

$$\alpha_\ell = \frac{\exp(s_\ell)}{\sum_{m=1}^L \exp(s_m)}. \quad (15)$$

Here, $\mathbf{W} \in \mathbb{R}^{d' \times d}$, $\mathbf{b} \in \mathbb{R}^{d'}$, and $\mathbf{w} \in \mathbb{R}^{d'}$ are learnable parameters of the attention network, and MLP_{ctx} is a multi-layer perceptron that transforms the weighted sum into the final context vector. The attention pooling allows the model to attend over the historical embeddings in a content-dependent manner and dynamically compute context u_c .

Token Composition. To implement Φ in Equation 12, we define a soft composition over a fixed set of candidate token embeddings $\{e_{c'}\}_{c' \in \mathcal{N}(c)}$, where we choose the candidate set $\mathcal{N}(c)$ of token c to be all tokens from same RQ-VAE codebook layer. Importantly, since $\mathcal{N}(c)$ includes all tokens within the same quantization layer, the token composition allows each token embedding to incorporate information from under-utilized or rarely selected codebook entries during pretraining. Our chosen candidate set enhances expressiveness by enabling the model to interpolate beyond the original static token assignments and leverage previously unused embedding capacity, effectively increasing the diversity of token representations. Empirically, the candidate set is chosen from the cached fused embeddings computed following Section 3.3. We then perform attention-based composition guided by the context vector u :

$$\alpha_{c'} = \frac{\exp(\langle W_q u, W_k e_{c'} \rangle)}{\sum_{c'' \in \mathcal{N}(c)} \exp(\langle W_q u, W_k e_{c''} \rangle)} \quad (16)$$

$$\tilde{e}_c = \Phi_{\text{soft}}(c, u_c, \mathcal{N}(c)) = \sum_{c' \in \mathcal{N}(c)} \alpha_{c'} \cdot e_{c'} \quad (17)$$

where $W_q, W_k \in \mathbb{R}^{d \times d}$ are learnable projection matrices. The context-aware composed token embedding $\tilde{e}_c$ is fused with the original static embedding e_c^{static} via a residual link:

$$e_{\text{final}} = \alpha \cdot \tilde{e}_c + (1 - \alpha) \cdot e_c^{\text{static}}, \quad \alpha \in [0, 1] \quad (18)$$

Here, α is a tunable hyperparameter that controls the strength of context adaptation, which is fixed throughout the training. Smaller α values prioritize the static embedding, while larger values encourage reinterpretation based on the extracted user interaction context. Overall, contextualized token composition enables the model to flexibly incorporate collaborative signals into token representations at generation time, addressing *suboptimal static tokenization* issue without modifying the tokenizer.

3.4.1 Learnable BOS Embedding Composition. For an RQ-VAE semantic indexing scheme, the first token in an item’s semantic ID typically captures coarse-grained, high-level semantics—such as the item’s broad category (e.g., “headphones” vs. “books”). As a result, any ambiguity or mis-assignment at the first token decoding step propagates to the whole sequence, since autoregressive generation interprets subsequent tokens relative to that prefix. Consequently, accurately modeling the first token is critical: it serves as a global semantic anchor that conditions downstream decoding and strongly influences the interpretation of subsequent tokens. To refine the first token generation, we extend the token composition mechanism by introducing a set of N learnable Beginning-of-Sequence (BOS)

query vectors that adaptively refine the initial semantic anchor to align with the user’s context, denoted as $\mathbf{Q}_{\text{BOS}} \in \mathbb{R}^{N \times d}$, where N is a hyperparameter controlling the number of BOS query vectors used. The learnable BOS query vectors serve as latent representations of a set of candidate BOS tokens and allow the model to perform contextual composition for the BOS token embeddings, providing tailored BOS token embeddings based on the input token sequences. Specifically, for the generation of a target token c , the BOS token is composed with contextual composition function Φ :

$$\hat{e}_{\text{BOS}} = \Phi(e_{\text{BOS}}, u_c, \mathbf{Q}_{\text{BOS}}), \quad (19)$$

where e_{BOS} is a zero vector. Therefore $\hat{e}_{\text{BOS}}$ is simply composed by BOS query vectors in $\mathbf{Q}_{\text{BOS}}$. The composed BOS embedding is used as the initial prefix for autoregressive generation of the item tokens, replacing the original BOS token. The learnable BOS queries form a unified scheme that ensures all generated token embeddings are dynamically adapted through contextualized composition. As a result, the model can better align the interpretation of each token with high-level semantic anchors (i.e., coarse-grained item semantics captured by the initial BOS composition), enabling more precise and coherent generation of semantic ID sequences.

3.5 Complexity Analysis

Decomposed Embedding Fusion introduces a constant per-item cost of $O(d^2)$ by projecting and combining token embeddings from pre-trained and collaborative sources. *Contextualized Token Composition* computes a context vector via attention pooling over L historical tokens, with complexity $O(L \cdot d^2)$. Each token composition attends to a fixed candidate set of size K , adding $O(K \cdot d^2)$ per token. Since d and K are constants, the additional cost scales linearly with context length L and remains controllable with a fixed candidate set compared to the $O(L^2 \cdot d)$ complexity of Transformer self-attention layers in the backbone recommender model.

We empirically evaluate the inference cost of DECOR against the baseline (TIGER [20]) without *Decomposed Embedding Fusion* or *Contextualized Token Composition*. Our results in Table 5 show that DECOR adds only 0.85 ms of additional latency per sample at a batch size of 128 when considering the full candidate set for token composition, while consistently improving retrieval accuracy by 5–14% over the base model.

3.6 DECOR Training

As shown in Figure 2, the proposed DECOR is integrated into every forward pass. We first apply decomposed embedding fusion to compute the encoder input embeddings by combining pretrained semantic embeddings with collaborative representations. During the autoregressive generation process, we replace the static embedding lookup with contextualized token composition, where each token embedding is dynamically adapted based on the generated context, which enables token embedding representations to evolve during training. To ensure semantic consistency, we reuse the fused vocabulary embeddings as the weights for the final prediction head. DECOR retains pretrained semantics and adapts to recommendation signals, effectively addressing both (C1) suboptimal static tokenization and (C2) discarded text semantics within a unified framework.

Table 1: Statistics of the Datasets

Dataset	#Users	#Items	#Interactions	Sparsity
Scientific	50,985	25,848	412,947	99.969%
Instrument	57,439	24,587	511,836	99.964%
Game	94,762	25,612	814,586	99.966%

4 Experiments

In this section, we conduct experiments on three real-world datasets of the most updated Amazon Review dataset [8] to investigate the efficacy of our method. We follow the commonly adopted evaluation protocol and data preprocessing as prior works [20, 24], and compare our method against a set of classic and state-of-the-art sequential recommendation baselines.

4.1 Experimental Setting

4.1.1 Dataset. To validate the effectiveness of our method, we follow the commonly adopted evaluation protocol as prior works [20, 24]. Specifically, we conduct experiments on three subsets of the most updated Amazon Review dataset [8]. We apply the 5-core filter preprocessing, excluding items and users with fewer than five interaction records. After that, we construct user interaction sequences by aligning the items chronologically, with the maximum item sequence length set to 20. Our processed dataset statistics are in Table 1. On all datasets, we evaluate all models using top-K Recall and NDCG with $K = \{5, 10\}$. Following standard practice [20], we adopt the leave-one-out strategy: for each user, the last interaction is used for testing, the second-last for validation, and the rest for training. We conduct a full-ranking evaluation over the entire candidate item set without sampling.

4.1.2 Implementation Details. For our method, we first implement and reproduce the reported performance of TIGER [20]. Specifically, we use Sentence-T5² as our text encoder for pretrained semantics and T5 as our generative recommender. Our experiments are carried out on a single NVIDIA Tesla A40 GPU. For reproducibility, we report the test set performance obtained with random seed 2025. To verify statistical significance, we conducted a paired t-test over five independent runs with seeds {2021, 2022, 2023, 2024, 2025}. In each run, the best model was selected based on validation set NDCG@10. For all experiments, we use a codebook size of 256 with 3 quantization levels that leads to 4-token semantic IDs.

4.2 Baseline Models

We compare our methods against a set of classic ID-based sequential recommenders and LLM-based generative recommenders. For generative recommenders, we re-run the official open-sourced code under identical experimental conditions to ensure fair comparison and to allow for training dynamics investigation. For other baselines that are originally trained under a different experimental setting, we adapt their official implementations to our experimental setting. And for baselines that are trained under the same experimental setting, we directly compare our performance with their reported performance in the paper. The traditional baselines are:

²<https://huggingface.co/sentence-transformers/sentence-t5-base>

Table 2: Performance comparison on three datasets: Instrument, Scientific, and Game. Metrics include Recall@K (R@K) and NDCG@K (N@K). Bold indicates the best result per column. Superscript * indicates statistical significance at $p < 0.05$.

Group	Method	Scientific				Instrument				Game			
		R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10
Traditional	Caser	0.0172	0.0281	0.0107	0.0142	0.0242	0.0392	0.0154	0.0202	0.0346	0.0567	0.0221	0.0291
	GRU4Rec	0.0221	0.0353	0.0144	0.0186	0.0345	0.0537	0.0220	0.0281	0.0522	0.0831	0.0337	0.0436
	SASRec	0.0256	0.0406	0.0147	0.0195	0.0341	0.0530	0.0217	0.0277	0.0517	0.0821	0.0329	0.0426
	BERT4Rec	0.0180	0.0300	0.0113	0.0151	0.0305	0.0483	0.0196	0.0253	0.0453	0.0716	0.0294	0.0378
	FDSA	0.0261	0.0391	0.0174	0.0216	0.0364	0.0557	0.0233	0.0295	0.0548	0.0857	0.0353	0.0453
	S ³ Rec	0.0253	0.0410	0.0172	0.0218	0.0340	0.0538	0.0218	0.0282	0.0533	0.0823	0.0351	0.0444
Generative	P5-SID	0.0155	0.0234	0.0103	0.0129	0.0319	0.0438	0.0237	0.0275	0.0480	0.0693	0.0333	0.0401
	P5-CID	0.0192	0.0300	0.0123	0.0158	0.0352	0.0507	0.0234	0.0285	0.0497	0.0748	0.0343	0.0424
	TIGER	0.0275	0.0431	<u>0.0181</u>	<u>0.0231</u>	0.0368	0.0574	0.0242	0.0308	0.0570	0.0895	0.0370	0.0471
	LETTER	<u>0.0276</u>	<u>0.0433</u>	0.0179	0.0230	0.0372	0.0581	0.0243	0.0310	0.0576	0.0901	0.0373	0.0475
	CoST	0.0270	0.0426	0.0180	0.0229	0.0366	0.0570	0.0242	0.0306	0.0569	0.0897	0.0379	0.0472
	ETEGRec	0.0272	<u>0.0433</u>	0.0173	0.0225	<u>0.0387</u>	<u>0.0609</u>	<u>0.0251</u>	<u>0.0323</u>	<u>0.0591</u>	<u>0.0925</u>	<u>0.0385</u>	<u>0.0492</u>
	DECOR	0.0309*	0.0469*	0.0206*	0.0257*	0.0409*	0.0617*	0.0272*	0.0339*	0.0610*	0.0944*	0.0400*	0.0507*

- **Caser** [22] applies narrow convolutional filters vertically across rows to learn sequential patterns and horizontally across columns to detect co-occurring latent features
- **SASRec** [12] uses a stack of Transformer encoder layers with multi-head self-attention to model long-range dependencies in user interaction sequences.
- **S³Rec** [31] enhances pre-training by introducing four auxiliary self-supervised tasks—masking attributes, predicting masked items, distinguishing subsequences, and contrasting full sequences—to maximize mutual information at multiple item granularities.
- **Bert4Rec** [21] employs a deep bidirectional Transformer encoder to learn rich, context-aware item representations for sequential recommendation.
- **GRU4Rec** [11] encodes session-based user behavior with a GRU network, then uses the final state to predict the next item.

The generative recommender baselines are:

- **TIGER** [20] frames sequential recommendation as a generative retrieval task by quantizing item text embeddings via RQ-VAE into a fixed vocabulary of semantic IDs and trains an LLM to autoregressively generate the next item’s ID.
- **LETTER** [24] optimizes an RQ-VAE tokenizer by enforcing contrastive alignment and diversity regularization to learn hierarchical, collaborative, and diverse item tokens.
- **P5-CID** [10] performs spectral clustering on collaborative co-occurrence graphs to group items, then uses the resulting cluster IDs as discrete tokens for generative recommendation.
- **P5-SID** [10] decomposes numeric item IDs into ordered subtokens (e.g., prefixes) so that frequently co-occurring or sequentially adjacent items share subtoken patterns, improving locality in autoregressive generation.

- **CoST** [32] trains a quantization codebook with an InfoNCE-style contrastive loss to map item embeddings into discrete semantic tokens that preserve both semantic similarity and neighborhood structure.
- **ETEGRec** [17] jointly optimizes the tokenizer and the recommender model, with a set of alignment losses to improve tokenizer-recommender consistency.

4.3 Overall Performance

As shown in Table 2, DECOR consistently outperforms all baselines across metrics and datasets. Compared to traditional models such as SASRec and FDSA, it achieves substantial gains by effectively integrating pretrained semantics and collaborative signals. Among generative methods, DECOR not only surpasses static-tokenization approaches (e.g., TIGER, LETTER, CoST) but also outperforms ETEGRec, a dynamic baseline that jointly optimizes the tokenizer and recommender under alignment objectives.

Quantitatively, DECOR delivers substantial improvements across all domains, achieving relative gains of more than 14% in NDCG@10 (Scientific), along with consistent improvements of 5% and 3% in Instruments and Games, respectively, over the strongest baselines. We specifically highlight the results on the Instrument dataset to illustrate the robustness of our approach. The instruments domain presents a unique challenge due to its reliance on specialized terminology (e.g., model numbers, tuning specs) rather than natural language descriptions. Consequently, generative baselines like TIGER encounter a performance bottleneck due to their reliance on static text priors. Similarly, the advanced dynamic tokenization baseline, ETEGRec, remains constrained as it attempts to align representations with these sparse pretrained semantic priors, failing to recover sufficient item distinctiveness. DECOR effectively circumvents this limitation. By decomposing the representation and explicitly fusing collaborative signals, our method compensates for the lack of

Table 3: Ablation results of DECOR on three datasets. Each variant incrementally adds components: decomposed embedding fusion (DEF), contextualized token composition (CTC), and learnable BOS queries (BOS). ✓ means the module is used, ✗ means not. Metrics include Recall@K (R@K) and NDCG@K (N@K). Bold indicates the best result per column.

Modules			Scientific				Instrument				Game			
DEF	CTC	BOS	R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10
✗	✗	✗	0.0275	0.0431	0.0181	0.0231	0.0368	0.0574	0.0242	0.0308	0.0570	0.0895	0.0370	0.0471
✗	✓	✗	0.0292	0.0459	0.0193	0.0247	0.0385	0.0595	0.0261	0.0321	0.0599	0.0931	0.0394	0.0502
✗	✓	✓	0.0300	0.0462	0.0198	0.0248	0.0397	0.0605	0.0263	0.0329	0.0600	0.0932	0.0395	0.0500
✓	✗	✗	0.0294	0.0457	0.0192	0.0246	0.0382	0.0583	0.0254	0.0323	0.0602	0.0934	0.0390	0.0501
✓	✓	✗	0.0298	0.0465	0.0198	0.0250	0.0388	0.0598	0.0257	0.0324	0.0603	0.0932	0.0396	0.0500
✓	✓	✓	0.0301	0.0469	0.0201	0.0256	0.0409*	0.0617*	0.0272*	0.0339*	0.0610	0.0944	0.0400	0.0507

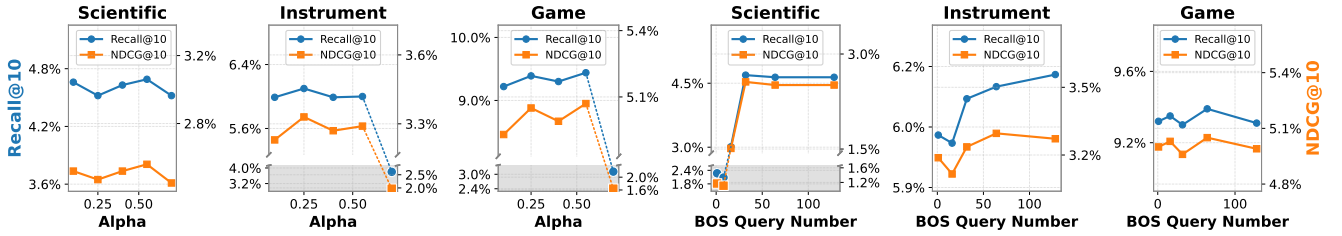


Figure 3: Parameter analysis of DECOR on contextual token composition weight α and BOS query number across all datasets. Shaded regions show performance collapse under extreme settings. We search the hyperparameters in the following region: $\alpha \in \{0.1, 0.25, 0.4, 0.55, 0.7\}$ and BOS Query Number $\in \{8, 16, 32, 64, 128\}$

semantic discriminability. The decomposed embedding fusion and contextualized decoding flow allow DECOR to capture fine-grained user preferences by adapting towards collaborative signals even when pretrained textual features are insufficient, securing a 5.0% improvement in NDCG@10 over the strongest baseline.

4.4 Ablation Study

Table 3 presents an ablation analysis of DECOR, evaluating the contribution of contextualized token composition (CTC), learnable BOS queries (BOS), and decomposed embedding fusion with pretrained embeddings (DEF). Starting from the base model (TIGER), both adding contextualized token composition-only and decomposed embedding fusion yield similarly significant improvements across all datasets, confirming the value of preserving pretrained semantics and contextualized token refinement in recommender training. The best performance is achieved when both pretrained semantics and context-aware token composition are used together. Interestingly, we observe that adding BOS queries introduces performance gains when used jointly with decomposed embedding fusion, which suggests that BOS queries act as high-level semantic anchors that strengthen the contextual alignment between pretrained semantics and the composed token representations. While CTC requires a set of input embeddings to refine against the user context, there is no preceding item token that exists at step 0; the learnable BOS queries provide a parametrized context prefix before decoding, making the initial token generation better adapt to user history. Our ablation result demonstrates the efficacy of BOS (e.g., -2.43% NDCG@10 Instruments). Overall, the full DECOR consistently outperforms all

ablations, demonstrating that each component contributes complementary benefits to the recommendation quality.

4.5 Hyperparameter Sensitivity

Figure 3 examines the impact of two key hyperparameters in DECOR: the composition weight α and the BOS query number (defined in Section 3.4). For the visualization of each hyper-parameter, we fix the other to the best-performed value in NDCG@10.

Effect of Composition Weight α . DECOR is generally robust to a range of α values. Moderate values (e.g., $\alpha = 0.4$ to 0.55) consistently yield the strongest performance across datasets, balancing contributions from the residual link and the context-aware composition. Note that even with $\alpha = 0.1$, we observe consistent improvements compared to the ablated baseline (w/ *Pretrained + Token Comp.* in Table 3). However, very high α values (e.g., 0.7) result in sharp performance degradation, likely due to undertraining of individual token embeddings caused by excessive reliance on compositional signals from other tokens. The collapse in convergence is particularly pronounced on the Instrument and Game datasets, where recommendation performance drops significantly. We observe that Instrument and Game have larger interaction spaces, requiring token embeddings to stabilize early to generalize across diverse usage contexts. Excessive reliance on composition in such cases can delay embedding convergence and lead to training collapse.

Effect of BOS Query Number. Under the best composition weight α , increasing the number of BOS queries consistently enhances performance, particularly when increasing from 0 to 32. On the Scientific dataset, for example, NDCG@10 improves from 1.17% to 2.56%, indicating that BOS queries facilitate meaningful convergence by enabling the model to better capture diverse user preferences before generating the first token. However, beyond 32 (or 64 on the Game dataset), performance gains plateau, and larger values such as 128 yield no additional improvement, which we attribute to the model having already captured sufficient contextual information with a moderate number of BOS queries. Additional BOS queries likely provide redundant signals that do not contribute further to recommendation quality.

4.6 Addressing Suboptimal Static Tokenization

To better understand whether our approach addresses suboptimal static tokenization, we present a case study on the Scientific dataset analyzing *prefix ambiguity*. In Figure 4, we present t-SNE visualizations of token embeddings before and after contextualized token composition. We observe that the static tokenization generates a single fixed embedding for the prefix (1, 276) due to deterministic embedding lookup, which lacks semantic coherence with valid next-token candidates (red triangles). In contrast, contextually composed prefix embeddings align more coherently with valid token candidates. The scattered composed prefix embeddings demonstrate that our method enhances prefix representations based on context, effectively mitigating the ambiguity of static tokenization.

A well-known challenge in quantized representation learning for recommender systems is that many codebook entries remain inactive, leading to wasted capacity and limited representation diversity [24]. For example, as shown in Table 4, only 25–28% of active embeddings at the first quantization layer are utilized during tokenizer pretraining across all datasets. In our experiments, we observe that DECOR improves the actively trained embedding coverage by involving inactive token embeddings in the token composition calculation, reaching 100% on Instrument and Game, and 51.06% on Scientific by filtering out tokens with below-uniform composition attention weights (Equation 16). Notably, on Scientific, which has 20–50% fewer interactions than the others (as shown in Table 1), DECOR activates fewer additional token embeddings to accommodate the less diverse contextual modeling, thereby mitigating suboptimal tokenization with efficient representation usage.

4.7 Comparison with Joint Tokenizer-Recommender Training

In Figure 5, we investigate the convergence behaviors on the Scientific dataset of DECOR and the tokenizer-recommender joint training baseline (ETEGRec [17]) by comparing their validation performance throughout the training session. Notably, direct run-time comparison is biased by implementations; therefore, we focus on sample efficiency as the robust metric. Figure 5 proves DECOR reaches 95% peak performance 23 epochs faster than ETEGRec. Importantly, ETEGRec exhibits learning stagnation during epochs 75–110, whereas DECOR improves stably, validating DECOR’s superior stability and sample efficiency compared to ETEGRec. In particular, we exclude the epochs where ETEGRec optimizes its tokenizers

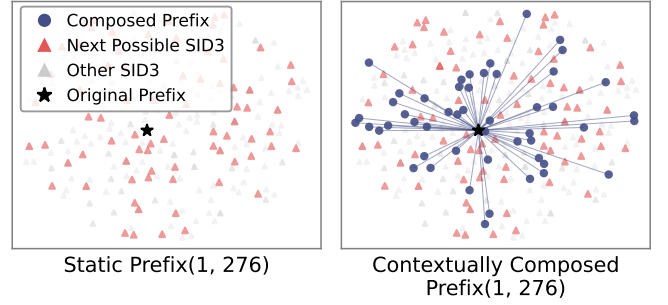


Figure 4: Case study for prefix ambiguity on the Scientific dataset. Compared to static tokenization (left), DECOR (right) produces prefix embeddings that are contextually adapted, enhancing expressiveness for disambiguation.

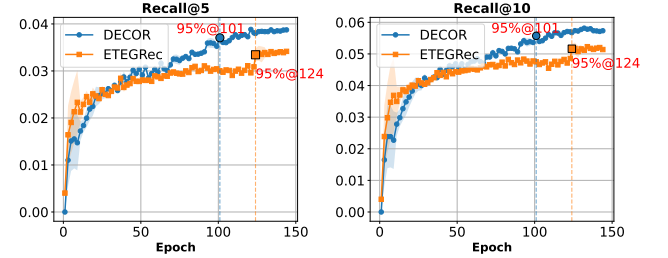


Figure 5: Convergence comparison of DECOR and ETEGRec on Recall@5 (left) and Recall@10 (right). DECOR not only achieves higher recall but also reaches 95% of its peak performance substantially earlier (101 vs. 124 epochs), indicating faster and more stable convergence.

and only report the validation results after every recommender update. We observe that while both models show improvements as the training progresses, DECOR consistently converges faster and to higher recall values. Specifically, DECOR reaches 95% of its best performance at around epoch 101, while ETEGRec requires 124 epochs to achieve the same threshold. Our results indicate that DECOR not only achieves better final accuracy but also requires fewer training epochs to stabilize compared to ETEGRec. In addition, we observe that the joint optimization of the tokenizer and recommender in ETEGRec adversely affects training stability. ETEGRec exhibits learning stagnation during epochs 75–110, whereas DECOR improves stably. Once the tokenizer is fixed in the final training stage, however, ETEGRec exhibits a noticeable performance gain, confirming that iterative co-optimization constrains the recommender’s learning due to constantly changing item token representations and validating DECOR’s superior stability and sample efficiency compared to ETEGRec.

4.8 Empirical Inference Cost Analysis

To empirically assess computational efficiency, we compare the generation inference cost with a beam search width of 50. As shown in Table 5, DECOR preserves the high efficiency of the underlying backbone. At a batch size of 128, the per-sample latency increases

Table 4: Comparison of active embedding usage at each quantization layer between the TIGER and DECOR.

Dataset	Method	Code Embedding Utilization		
		Layer-1	Layer-2	Layer-3
Scientific	TIGER	26.6%	99.07%	99.87%
	Ours	51.06%	99.97%	100.00%
Instrument	TIGER	27.97%	96.77%	100.00%
	Ours	100.00%	100.00%	100.00%
Game	TIGER	25.67%	99.61%	100.00%
	Ours	100.00%	100.00%	100.00%

Table 5: Inference efficiency comparison using Beam Search ($k = 50$). Both methods demonstrate stable performance. DECOR introduces minimal latency overhead (<1 ms) while maintaining $\approx 82\%$ of the baseline throughput.

Batch	TIGER (Baseline)		DECOR (Ours)		Cost
	Lat. (ms)	Thr. (s/s)	Lat. (ms)	Thr. (s/s)	Δ ms
8	3.72	269	4.49	223	+0.77
16	3.68	272	4.49	223	+0.81
32	3.82	262	4.65	215	+0.83
64	3.83	261	4.67	214	+0.84
128	3.85	260	4.70	213	+0.85

marginally from 3.85 ms (TIGER) to 4.70 ms (DECOR), representing a constant overhead of approximately 0.85 ms, which is primarily attributed to the fixed memory access patterns in the fusion module rather than computational complexity scaling of token composition. DECOR maintains a robust throughput of >210 samples/second, retaining approximately 82% of the baseline’s speed. Given the significant accuracy improvements of 5–14% observed in Table 2, this sub-millisecond latency cost represents a highly favorable trade-off, confirming that DECOR is a cost-effective solution well-suited for real-time deployment.

5 Conclusion

In this work, we address two limitations of existing generative recommenders: the suboptimal static tokenization and the discarded pretrained semantics. We propose DECOR, a unified framework that enhances token adaptability through contextualized token composition and preserves pretrained knowledge via decomposed embedding fusion. Experiments on three real-world datasets confirm that DECOR outperforms state-of-the-art baselines with higher accuracy and moderate computational cost, while ablation studies highlight DECOR’s ability to bridge pretrained semantics and user-behavior dynamics in generative recommendation.

6 Acknowledgement

This research is supported in part by the National Science Foundation under Grant No. CNS-2427070, IIS-2331069, IIS-2202481, IIS-2130263, CNS-2131622. The views and conclusions contained in this document are those of the authors and should not be interpreted as

representing the official policies, either expressed or implied, of the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation here on.

References

- [1] Keqin Bao, Jizhi Zhang, Wenjie Wang, Yang Zhang, Zhengyi Yang, Yanchen Luo, Chong Chen, Fuli Feng, and Qi Tian. 2025. A bi-step grounding paradigm for large language models in recommendation systems. *ACM Transactions on Recommender Systems* 3, 4 (2025), 1–27.
- [2] Zhixuan Chu, Hongyan Hao, Xin Ouyang, Simeng Wang, Yan Wang, Yue Shen, Jinjie Gu, Qing Cui, Longfei Li, Siqiao Xue, et al. 2023. Leveraging large language models for pre-trained recommender systems. *arXiv preprint arXiv:2308.10837* (2023).
- [3] Sunhao Dai, Ninglu Shao, Haiyuan Zhao, Weijie Yu, Zihua Si, Chen Xu, Zhongxiang Sun, Xiao Zhang, and Jun Xu. 2023. Uncovering chatgpt’s capabilities in recommender systems. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 1126–1132.
- [4] Yashar Deldjoo, Zhankui He, Julian McAuley, Anton Korikov, Scott Sanner, Arnau Ramisa, René Vidal, Maheswaran Sathiamoorthy, Atoosa Kasirzadeh, and Silvia Milano. 2024. A Review of Modern Recommender Systems Using Generative Models (Gen-RecSys). In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Barcelona, Spain) (KDD '24)*. Association for Computing Machinery, New York, NY, USA, 6448–6458. doi:10.1145/3637528.3671474
- [5] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. Onerec: Unifying retrieve and rank with generative recommender and iterative preference alignment. *arXiv preprint arXiv:2502.18965* (2025).
- [6] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM conference on recommender systems*. 299–315.
- [7] Yupeng Hou, Zhankui He, Julian McAuley, and Wayne Xin Zhao. 2023. Learning Vector-Quantized Item Representation for Transferable Sequential Recommenders (*WWW '23*). Association for Computing Machinery, New York, NY, USA, 1162–1171. doi:10.1145/3543507.3583434
- [8] Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiushi Chen, and Julian McAuley. 2024. Bridging Language and Items for Retrieval and Recommendation. *arXiv preprint arXiv:2403.03952* (2024).
- [9] Yupeng Hou, Jianmo Ni, Zhankui He, Naveen Sachdeva, Wang-Cheng Kang, Ed H Chi, Julian McAuley, and Derek Zhiyuan Cheng. 2025. Actionpiece: Contextually tokenizing action sequences for generative recommendation. *arXiv preprint arXiv:2502.13581* (2025).
- [10] Wenyue Hua, Shuyuan Xu, Yingqiang Ge, and Yongfeng Zhang. 2023. How to index item ids for recommendation foundation models. In *Proceedings of the Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*. 195–204.
- [11] Dietmar Jannach and Malte Ludewig. 2017. When recurrent neural networks meet the neighborhood for session-based recommendation. In *Proceedings of the eleventh ACM conference on recommender systems*. 306–310.
- [12] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.
- [13] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. 2022. Autoregressive image generation using residual quantization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 11523–11532.
- [14] Yongqi Li, Nan Yang, Liang Wang, Furu Wei, and Wenjie Li. 2023. Generative retrieval for conversational question answering. *Information Processing & Management* 60, 5 (2023), 103475.
- [15] Jiayi Liao, Sihang Li, Zhengyi Yang, Jiancan Wu, Yancheng Yuan, and Xiang Wang. 2023. Llora: Aligning large language models with sequential recommenders. *CoRR* (2023).
- [16] Jiayi Liao, Sihang Li, Zhengyi Yang, Jiancan Wu, Yancheng Yuan, Xiang Wang, and Xiangnan He. 2024. Llora: Large language-recommendation assistant. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1785–1795.
- [17] Enze Liu, Bowen Zheng, Cheng Ling, Lantao Hu, Han Li, and Wayne Xin Zhao. 2025. Generative Recommender with End-to-End Learnable Item Tokenization. arXiv:2409.05546 [cs.LG]. <https://arxiv.org/abs/2409.05546>
- [18] Yifan Liu, Yaokun Liu, Zelin Li, Ruichen Yao, Yang Zhang, and Dong Wang. 2025. Modality Interactive Mixture-of-Experts for Fake News Detection. In *Proceedings of the ACM on Web Conference 2025 (Sydney NSW, Australia) (WWW '25)*. Association for Computing Machinery, New York, NY, USA, 5139–5150. doi:10.1145/3696410.3714522

- [19] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* 21, 140 (2020), 1–67. <http://jmlr.org/papers/v21/20-074.html>
- [20] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Keshavan, Trung Vu, Lukasz Heidt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. 2023. Recommender systems with generative retrieval. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '23). Curran Associates Inc., Red Hook, NY, USA, Article 452, 17 pages.
- [21] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.
- [22] Jiayi Tang and Ke Wang. 2018. Personalized top-n sequential recommendation via convolutional sequence embedding. In *Proceedings of the eleventh ACM international conference on web search and data mining*. 565–573.
- [23] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. 2017. Neural discrete representation learning. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, 6309–6318.
- [24] Wenjie Wang, Honghui Bao, Xinyu Lin, Jizhi Zhang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Learnable Item Tokenization for Generative Recommendation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management* (Boise, ID, USA) (CIKM '24). Association for Computing Machinery, New York, NY, USA, 2400–2409. doi:10.1145/3627673.3679569
- [25] Yidan Wang, Zhaochun Ren, Weiwei Sun, Jiyuan Yang, Zhixiang Liang, Xin Chen, Ruobing Xie, Su Yan, Xu Zhang, Pengjie Ren, et al. 2024. Enhanced generative recommendation via content and collaboration integration. *CoRR* (2024).
- [26] Ye Wang, Jiahao Xun, Minjie Hong, Jieming Zhu, Tao Jin, Wang Lin, Haoyuan Li, Linjun Li, Yan Xia, Zhou Zhao, et al. 2024. Eager: Two-stream generative recommender with behavior-semantic collaboration. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3245–3254.
- [27] Jun Yin, Zhengxin Zeng, Mingzheng Li, Hao Yan, Chaozhuo Li, Weihao Han, Jianjin Zhang, Ruochen Liu, Hao Sun, Weiwei Deng, Feng Sun, Qi Zhang, Shirui Pan, and Senzhang Wang. 2025. Unleash LLMs Potential for Sequential Recommendation by Coordinating Dual Dynamic Index Mechanism. In *THE WEB CONFERENCE 2025*. <https://openreview.net/forum?id=GE71TxvTH3>
- [28] Junjie Zhang, Ruobing Xie, Yupeng Hou, Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2025. Recommendation as instruction following: A large language model empowered recommendation approach. *ACM Transactions on Information Systems* 43, 5 (2025), 1–37.
- [29] Yuhui Zhang, Hao Ding, Zeren Shui, Yifei Ma, James Zou, Anoop Deoras, and Hao Wang. 2021. Language models as recommender systems: Evaluations and limitations. (2021).
- [30] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting Large Language Models by Integrating Collaborative Semantics for Recommendation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. 1435–1448. doi:10.1109/ICDE60146.2024.00118
- [31] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. 2020. S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. In *Proceedings of the 29th ACM international conference on information & knowledge management*. 1893–1902.
- [32] Jieming Zhu, Mengqun Jin, Qijiong Liu, Zexuan Qiu, Zhenhua Dong, and Xiu Li. 2024. CoST: Contrastive Quantization based Semantic Tokenization for Generative Recommendation. arXiv:2404.14774 [cs.LG] <https://arxiv.org/abs/2404.14774>